XThreat: Explainable and Evidence-Aware AI for Cyber Threat Detection

Abstract

A network-intrusion alert that only says "attack detected" gives a security analyst no basis for trusting, triaging or acting on it. The course project therefore prescribed a seven-stage report (*attack detected → why → evidence → important features → attack behaviour → confidence → recommended response*). This report documents how the instructor's pipeline was implemented on CIC-IDS2017 (2.83 M flows, 15 classes) in a primary notebook, how it was revised into a third version (v3) after five AI review reports, and exactly where the two pipelines differ. XGBoost reaches 99.82 % accuracy and a macro-F1 of 0.834 on 495,192 held-out flows (benign-versus-attack F1 0.996, false-alarm rate 0.15 %); under a per-class chronological split macro-F1 falls to 0.723. The v3 experiments, all executed, show that the tree ensembles are not statistically separable (three-seed macro-F1: XGBoost 0.845, Random Forest 0.824, LightGBM 0.810) while Logistic Regression and an FT-style Transformer (0.670) are clearly worse; 51–74 % of random-split test flows have a near-identical training flow; capped SMOTE showed no benefit in a single-factor ablation; probabilities are already well calibrated (ECE 0.0005) and conformal sets flag 0.12 % of flows for analyst review; SHAP and LIME agree above chance but weakly (overlap@5 0.273 versus a chance level of 0.167) and SHAP is the more faithful (AOPC 0.806 versus 0.589); constrained counterfactuals change 2 instead of 10 features. The cross-dataset test was not run.

Keywords: intrusion detection, explainable AI, SHAP, LIME, counterfactual explanation, calibration, conformal prediction, CIC-IDS2017.

1. Introduction

Flow-level classifiers trained on benchmarks such as CIC-IDS2017 [1] routinely exceed 99 % accuracy, and gradient-boosted trees are among the strongest and cheapest models for tabular data [2–4]. In a Security Operations Centre (SOC), however, an alert is useful only if the analyst can decide whether to believe it, how to prioritise it and what to do next [5–7]; analyst studies report that contextual, evidence-backed explanations matter more than raw outputs [8, 9]. Explainable-AI methods supply building blocks (Shapley attributions, local surrogates, counterfactuals), but two gaps remain: explanations usually stop at a feature-importance plot instead of an evidence chain, and the explanations themselves are seldom tested [10–12].

Objectives. (1) Implement the instructor's pipeline end to end on CIC-IDS2017 (primary notebook); (2) evaluate detection honestly with a leakage-aware protocol, an alternative chronological split and over-fitting checks; (3) produce the seven-stage analyst report from SHAP, LIME and counterfactual outputs; (4) review the first implementation, plan improvements from five AI review reports and implement them in a final notebook (v3); (5) document stage by stage how the final pipeline differs from the reference pipeline.

Notebooks. The *primary notebook* (`C233188_CodeNotebook.ipynb`, 90 cells) implements the instructor's pipeline. The *updated notebook* (`C233188_Yamin3_v3.ipynb`, 135 cells) contains the same Sections 1–20 plus Section 21 (v3 experiments) and Section 22 (publication figures). **Structure:** Sections 2–3 review the literature and related work, Section 4 defines the methodology, Section 5 compares the pipelines and notebooks, Sections 6–8 present data, implementation and results, and Sections 9–11 discuss, state limitations and conclude.

2. Literature Review

Learning-based intrusion detection. Network intrusion detection moved from signature- and rule-based systems to anomaly detection and then to supervised and deep learning [13, 14]. Sommer and Paxson [15] argued early that machine learning for intrusion detection differs fundamentally from classification in other domains because errors are asymmetric, traffic is non-stationary and evaluation data rarely

resemble deployment. Arp et al. [16] catalogued ten recurring pitfalls (sampling bias, label inaccuracy, data snooping, spurious correlations, inappropriate baselines) in security machine learning. Three of them are central here: data snooping, spurious correlation (a file-of-origin shortcut found in an earlier configuration) and sampling bias (random splits of near-duplicate flows).

Datasets and their quality. CIC-IDS2017 [1] contains labelled CICFlowMeter statistics for benign and attack traffic over five days and is among the most used flow benchmarks [17–19]. Re-analyses found flow-construction errors, mislabelled flows and artefacts that change measured performance [20, 21], and the released CSV files lack timestamps and IP addresses. These findings motivate the cleaning stages and the leakage-aware protocol of this project.

Class imbalance. Benign traffic dominates, and the rarest attack classes (11–36 flows here) are those an analyst may care about most. Learning from imbalanced data is a long-standing topic [22]; common remedies are synthetic oversampling such as SMOTE [23, 24], cost-sensitive weighting and undersampling. Oversampling a handful of real points creates synthetic examples that add no new information, and macro-averaged metrics [25] are fragile when a class has a few test flows, so supports and uncertainty intervals must be reported.

Tree ensembles versus deep models. Gradient-boosted trees [2, 3] and random forests remain very competitive on tabular data; Grinsztajn et al. [4] and Shwartz-Ziv and Armon [26] report that tree models often outperform deep networks on medium-sized tabular benchmarks. Transformers [27] have been adapted by tokenising each feature (FT-Transformer [28]), whereas graph neural networks [29, 30] need relational structure such as host-to-host connections [31]. Hence tree ensembles are the main detectors here, a feature-token Transformer is a paired comparison, and a GNN is future work because the CSV release has no addressing information.

Explainable AI: methods and reliability. Explainability aims to make decisions understandable to a stakeholder [5, 6, 32]; Miller [7] argues that human explanations are contrastive, which supports counterfactual statements such as "the alert would not have fired if …". SHAP distributes a prediction fairly over features and is exact for trees [33, 34]; LIME fits a local sparse surrogate [35]; counterfactual explanations search for a minimal change that flips the decision [36, 37]; Integrated Gradients attributes a neural output along a path from a baseline and satisfies a checkable completeness property [38]. Explanations are not automatically trustworthy: they can be manipulated [39], are unstable [40] and disagree [10]. Faithfulness is commonly tested by perturbation, i.e. removing the top-ranked features should reduce the prediction more than removing random ones, summarised by the area over the perturbation curve (AOPC) [41].

Uncertainty and evaluation statistics. A maximum softmax probability is often reported as "confidence", but classifiers can be miscalibrated [42]; temperature scaling, ECE and the Brier score address this. Conformal prediction [43, 44] produces prediction sets that contain the true label with a chosen probability under exchangeability: a singleton set can be auto-accepted, a multi-label set referred to an analyst. Because some test classes have very few flows, the bootstrap [45] gives confidence intervals (paired when comparing two models), McNemar's test [46] compares paired predictions and the Matthews correlation coefficient [47] summarises a confusion matrix robustly to imbalance.

Gaps. The literature leaves four gaps addressed here: (i) explanations that stop at a plot rather than an analyst-readable evidence chain; (ii) explanation methods applied but not tested for agreement, fidelity or necessity; (iii) a "confidence" that is an uncalibrated softmax; (iv) detection results without uncertainty, near-duplicate control or temporal split.

3. Related Work

Early explainable-IDS studies mostly present feature-importance plots for a trained detector [48–51]; Warnecke et al. [52] showed that explanation methods differ substantially in fidelity on security models. Recent studies begin to evaluate explanations but each isolates one aspect on a different dataset or model family (Table 1). Dietz et al. [11] and Kumar and Thing [12] find that agreement between explainers varies from high to low (0.0–0.38 for TRUSTEE versus SHAP), Hermosilla et al. [53] find SHAP and LIME complementary, and Kalakoti et al. [54] obtain only moderate faithfulness correlations on real SOC alerts. Galwaduge and Samarabandu [55] show that unconstrained counterfactual baselines perform poorly in validity and plausibility. On evaluation protocol, Moczkodan and Ragab [56] report that split protocol and padding matter more than architecture on CIC-IDS2017 sequences and that a plain time-based split is degenerate, while Srisumrith and Sodsee [57] report macro-F1 up to 0.997 under random splits without evaluating the explanations. To our knowledge none of these combines, on the same alerts, an agreement audit, a fidelity and necessity test and a shortcut check with a generated seven-stage report; v3 further adds calibrated confidence, conformal sets and a paired statistical protocol. Numbers across studies are not directly comparable (different datasets and metrics).

Table 1. Recent related studies (2022–2026) compared with this work.

Study	Data / models	Explanation focus	Reported finding
Srisumrith and Sodsee [57]	CIC-IDS2017 (20 % sample); XGBoost, RF, LR	SHAP (RF); leakage screening	Macro-F1 up to 0.997 (random split); explanations not evaluated
Moczkodan and Ragab [56]	CIC-IDS2017 sequences; 9 models	None (split / padding protocol)	Random vs. group split changes ranking; RF most robust (0.778)
Dietz et al. [11]	Several NIDS datasets	SHAP, LIME, IG, DeepLIFT, others	Consensus between methods varies from high to low
Kumar and Thing [12]	Kitsune, HorusEye, ENIDrift, HELAD	TRUSTEE vs. SHAP	Agreement 0.0–0.38 depending on model
Hermosilla et al. [53]	UNSW-NB15; XGBoost, TabNet	SHAP and LIME	Complementary; XGBoost more stable (Jaccard 0.82 vs. 0.65)
Kalakoti et al. [54]	Real SOC alerts; LSTM	LIME, SHAP, IG, DeepLIFT	Faithfulness: SHAP 0.40, LIME 0.42, DeepLIFT 0.76
Galwaduge and Samarabandu [55]	UNSW-NB15, CIC-DDoS-2019, CIC-IDS2017	Diffusion counterfactuals	High validity and plausibility; unconstrained baselines poor
Eriksson and Grov [8], Rastogi et al. [9]	SOC analyst interviews / survey	Explanation needs	Analysts value contextual, role-aware explanations
This work	CIC-IDS2017; XGBoost, LightGBM, RF, LR, FT-Transformer	SHAP, LIME, constrained CF, IG + evidence layer	Agreement vs. chance, fidelity, stability, port ablation, calibration, conformal sets, paired bootstrap

4. Methodology

Overview. XThreat has four parts (Figures 1–2): (A) leakage-aware data preparation, (B) detection models, (C) explanation and evidence components and (D) the seven-stage analyst report. The detection half (A, B) and the explanation half (C, D) share the same trained model and feature space, so every explanation refers to the features the model used. The updated pipeline adds a statistical validation layer (E).

Problem formulation. Let $D = \{(x_i, y_i)\}$ be flow records with $x_i \in R^d$ and $y_i \in \{1,\ldots,K\}$, $K = 15$. The detector returns class probabilities $p_k(x)$; the decision is $\hat{y} = \text{argmax}_k\, p_k(x)$ and, in the primary version, the confidence is $\max_k p_k(x)$. For each alert ($\hat{y} \neq$ BENIGN) the system outputs (why, evidence, top features, behaviour, confidence, response).

Leakage rules and splits. Every statistic that depends on the data distribution (imputer medians, scaler parameters, mutual-information ranking, SMOTE neighbours) is estimated on the training partition only; row-wise, parameter-free transformations (log(1+x), ratios) are applied before the split; the test set is never resampled; provenance variables are excluded. Three splits are used: a stratified 80/20 random split

(main results), a per-class chronological split (first 80 % of each class in capture order for training) and, in v3, a group-aware split that keeps near-duplicate flows together.

Detection layer. XGBoost fits an additive tree ensemble with softmax outputs and per-sample weights [2]; LightGBM uses histogram-based, leaf-wise growth with early stopping [3]; Random Forest and Logistic Regression are baselines. In v3 an FT-Transformer-style model [28] embeds each of the 30 features as a token, applies self-attention [27] and classifies from a class token.

Explanation layer. *SHAP:* the Shapley value of a feature is its average marginal contribution over feature subsets, giving $f(x) = \varphi 0 + \Sigma\ \varphi_j(x)$ [33]; TreeSHAP is exact for trees [34]; values are in log-odds, one set per class, and global importance is the mean $|\varphi_j|$. *LIME:* a weighted sparse linear surrogate is fitted around the alert [35]; weights are in probability units and features are quantile-binned. *Counterfactual:* the primary notebook uses a greedy search that moves the $k = 10$ features with the largest gap towards the benign mean by a fraction $\alpha = 0.2$ per iteration until the arg-max flips [36]; it has no plausibility constraint. The v3 search freezes `Destination Port`, clips features to the 1–99 % benign range, prunes to a minimal change set and checks plausibility by kNN distance to benign flows. *Integrated Gradients* [38] with the benign mean as baseline explains the Transformer; completeness requires $\Sigma\ IG_j = f(x) - f(\text{baseline})$.

Evidence layer. A deterministic mapping from model output to analyst language: (i) a dictionary from feature names to plain meanings; (ii) templates generating "abnormally high/low *meaning*" from the SHAP sign and the position relative to the mean benign flow, and naming the destination port as a service ("targets port 21 (FTP)"); (iii) per-class dictionaries of attack-behaviour narratives and recommended responses for all 15 classes. Rule-based generation is deliberate: every sentence is traceable to a feature, a SHAP value or a template, so the report cannot hallucinate.

v3 validation methodology. *Uncertainty:* a paired bootstrap (1,000 resamples of test rows) gives CIs for macro-F1 and for differences from XGBoost [45]; McNemar's test [46], MCC [47] and balanced accuracy complement accuracy; three seeds (11, 22, 33) re-draw the split, resampling, feature selection and initialisation. *Near-duplicates:* a test flow is a near-duplicate if its 30 selected log-scale features, rounded to 1 or 0 decimals, equal those of a training flow; a GroupShuffleSplit over these keys gives the group-aware split. *Imbalance ablation:* natural unweighted data versus capped sqrt-inverse-frequency weights versus capped SMOTE, everything else fixed. *Calibration and conformal sets:* ECE, Brier and NLL with temperature scaling [42] on a 30 % calibration part of the test flows; split-conformal sets at $\alpha = 0.05$ with score $1 - p_y(x)$ [44], marginal and class-conditional (Mondrian, falling back to the marginal threshold for classes with fewer than 20 calibration flows). *Explanation audit* on 55 predicted-attack flows: overlap@k against chance k/30, Spearman correlation, cluster-level overlap (13 clusters of features with $|r| > 0.9$), deletion fidelity (AOPC [41]) and sufficiency against a random ranking, SHAP stability under Gaussian noise ($\sigma = 0.1$) and LIME self-consistency over two seeds.

5. Instructor's Pipeline versus the Updated Pipeline

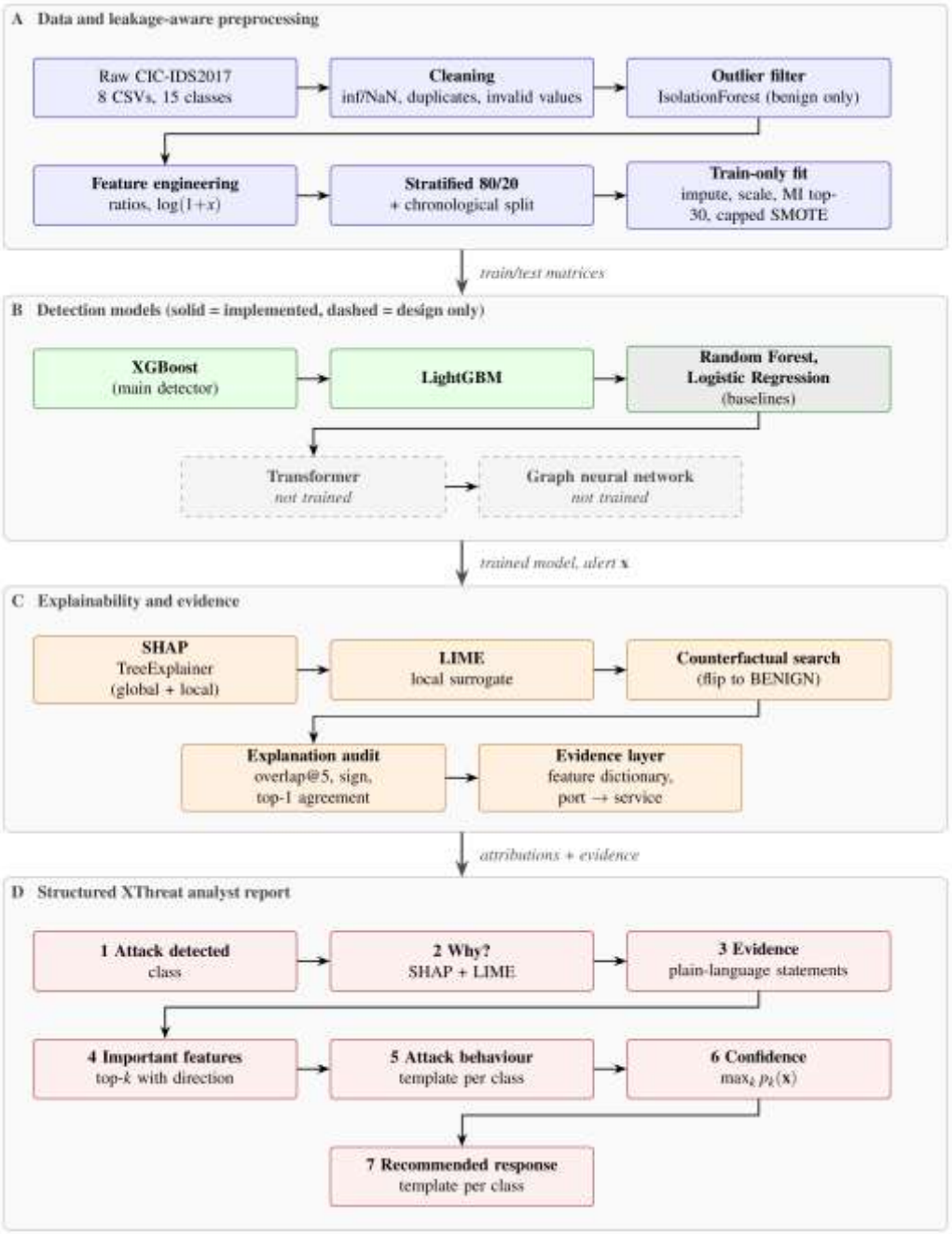


Figure 1. Pipeline of the primary implementation (built on the instructor's pipeline). Part D is the instructor's seven-stage report: attack detected → why → evidence → important features → attack behaviour → confidence → recommended response.

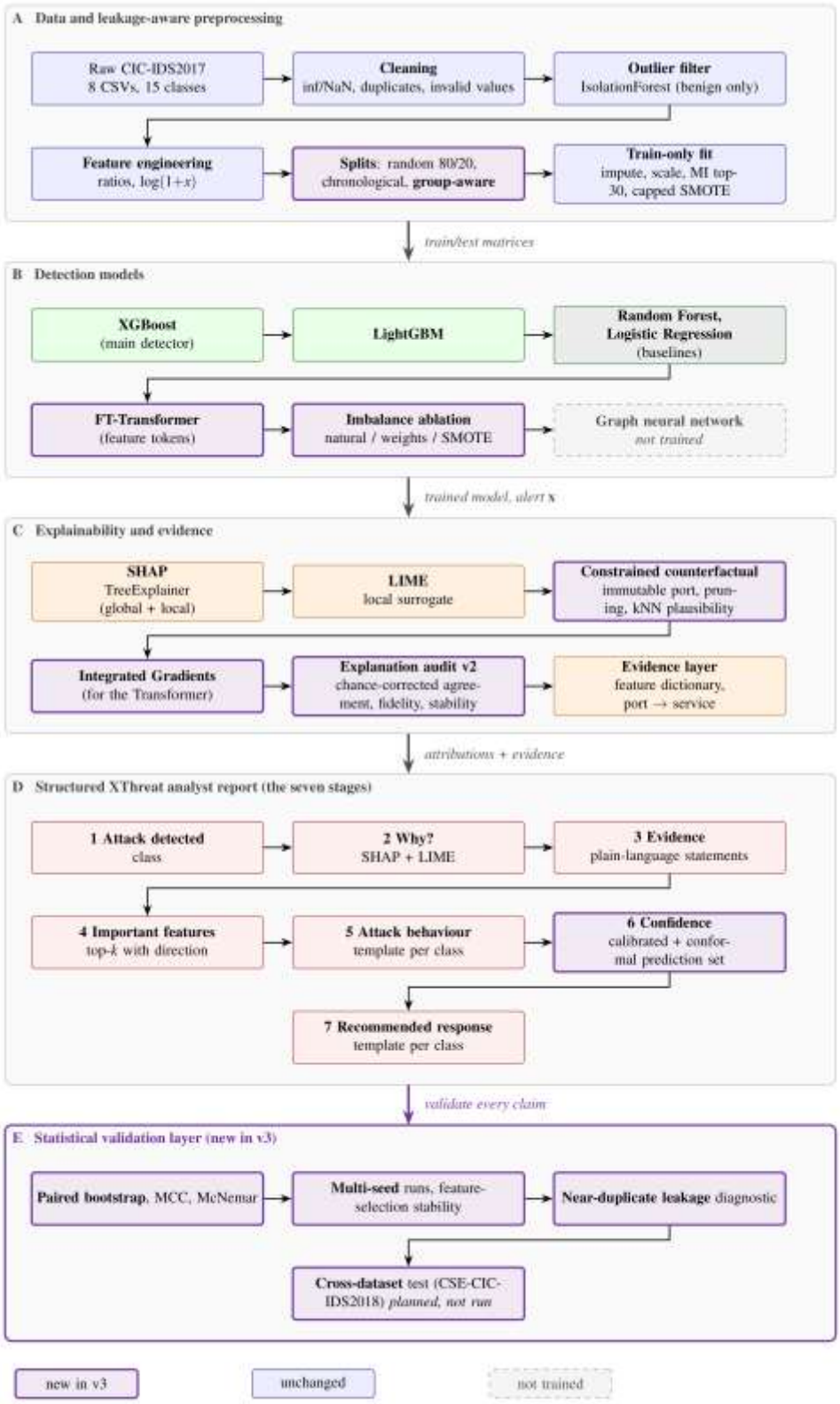


Figure 2. Updated pipeline (v3). Parts A–D keep the primary structure and the seven stages; purple boxes are new in v3: group-aware split, imbalance ablation, FT-Transformer with Integrated Gradients, constrained counterfactuals, explanation audit v2, calibrated confidence with conformal sets, and a statistical validation layer (E). The GNN remains design-only because the CSV release has no IP addresses; the cross-dataset test was planned but not run.

5.1 Stage-by-stage comparison

Table 2. Instructor's seven report stages: primary implementation versus updated (v3) pipeline.

Stage (reference)	Primary implementation	Updated v3 pipeline	Change
1 Attack detected	Argmax of XGBoost probabilities (15 classes)	Same; paired-bootstrap CIs, MCC, McNemar, multi-seed runs	Evaluation only
2 Why?	SHAP and LIME on the predicted class	Same; chance-corrected SHAP–LIME agreement, Spearman, cluster-level agreement	Audit added
3 Evidence	Rule-based sentences from feature dictionary and SHAP sign; port → service	Same templates; backed by deletion/sufficiency fidelity and stability tests	Audit added
4 Important features	Top-k by |SHAP| with direction	Same; checked against random ranking and Integrated Gradients	Audit added
5 Attack behaviour	Per-class narrative templates	Unchanged	None
6 Confidence	Raw softmax max $p_k(x)$	Calibration (ECE, Brier, NLL, temperature) and split-conformal sets; singleton = auto-accept, multi-label = analyst	**Method changed**
7 Response	Per-class templates	Unchanged	None
Counterfactual (2–4)	Greedy, unconstrained	Immutable port, benign bounds, pruning, kNN plausibility	**Method changed**
Models	XGBoost, LightGBM, RF, LR; Transformer/GNN stubs not run	Adds FT-Transformer with paired protocol; GNN design-only	Model added

5.2 Notebook comparison and update plan

Both notebooks were compared cell by cell (source and saved outputs). **Sections 1–20 are functionally identical** and give identical numbers (XGBoost macro-F1 0.8336, chronological 0.7232, SHAP–LIME overlap@5 0.31); v3 only adds technique-family tags to section titles, explanatory notes, comments and a "Why? → Evidence" label in the report cell. v3 is append-only: Section 21 (cells 21.0–21.10, nine experiment blocks) and Section 22 (figure set A–M) bring the notebook from 90 to 135 cells. The five AI review reports were consolidated into the concern table of the v3 notebook, reproduced in Table 3 with the answer and status of each item.

Table 3. Update plan: reviewer concern, v3 change and evidence status.

Cell	Concern about primary version	v3 answer	Status
21.1	Single run, no uncertainty or significance test	Paired bootstrap CIs, MCC, McNemar	Executed
21.2	Conclusions may depend on one seed	3-seed splits, mean ± SD, t-interval, selection stability	Executed
21.3	Random split inflated by near-duplicates	Near-duplicate rate, group-aware split	Executed
21.4	SMOTE vs. weights never isolated	Single-factor imbalance ablation	Executed
21.5	"Confidence" is an uncalibrated softmax	ECE/Brier/NLL, temperature, conformal sets	Executed
21.6	Agreement without chance, fidelity, stability	Chance-corrected agreement, deletion fidelity, stability	Executed
21.7	Counterfactuals unconstrained, unvalidated	Immutable port, bounds, pruning, plausibility	Executed
21.8	Transformer required but missing	FT-Transformer, paired comparison, Integrated Gradients	Executed
21.9	Single dataset	Zero-shot test on CSE-CIC-IDS2018	**Not run** (no 2018 data)
22	No publication-quality figures	Figures A–M (L skipped)	Executed

Deliverables covered: instructor's pipeline (Figure 1, Table 2); primary implementation (Sections 6–8); update plan (Table 3); final implementation (Figure 2, Section 8.2); explanation (this report).

6. Dataset and Preprocessing

CIC-IDS2017 [1] (`MachineLearningCSV`) holds 2,830,743 flows in eight CSV files with 78 numeric features and a label. Attacks are scheduled by day (FTP/SSH-Patator Tuesday; DoS and Heartbleed Wednesday; web attacks and infiltration Thursday; Bot, PortScan and DDoS Friday). Benign traffic is 80.3 % of rows; Heartbleed (11), SQL injection (21) and Infiltration (36) are practically negligible. The preprocessing chain follows the instructor's technique families (Table 4).

Table 4. Preprocessing steps and row counts.

Step (family)	Action	Rows after
Merge 8 CSV files	Concatenate; 4,376 infinite and 1,358 missing values found	2,830,743
Missing data (1)	±∞ → NaN; median imputation (0.10 % of values); 40 % column-drop rule never triggered	2,830,743
Cleaning (2)	Duplicate column dropped; labels repaired to 15 classes; 331,200 duplicate rows (11.70 %) removed	2,499,543
Invalid values (2)	2,881 flows with negative time/rate values removed (0.115 %)	2,496,662
Outliers (3)	IQR/z-score as diagnostics only; IsolationForest [58] on benign rows only removes 20,705 (1 % of benign)	2,475,957
Features (9, 8, 6)	5 engineered features (82 candidates); label encoding; log(1+x) on 64 skewed features	2,475,957
Split (train/test)	Stratified 80/20 (seed 42); chronological 80/20 per class	1,980,765 / 495,192
Train-only fit (10)	Median imputer, StandardScaler, mutual-information top-30 features [59]	–
Imbalance (4, 5)	Capped SMOTE: classes with ≥50 and <10,000 training samples raised to 10,000 (52,720 synthetic rows); Heartbleed, Infiltration, SQL injection get weight min(50, 10,000/n) = 50	2,033,485 (train)

Statistical analysis (Family 12) showed that effect sizes, not p-values (numerically zero at this sample size), are informative: the largest are Bwd Packet Length Mean (Cohen's d = 1.31), Flow Duration (0.70) and Flow IAT Mean (0.54). Oversampling every class to the benign size would have created over 1.6 M synthetic rows from six real Heartbleed flows, hence the capped policy. The chronological split orders flows within each class by weekday and row order (a "train Monday–Thursday, test Friday" design is impossible because PortScan, DDoS and Bot occur only on Friday); it assumes that row order approximates capture time. A file-of-origin code (`Day_Index`) used in an even earlier configuration was removed as a provenance shortcut.

7. Implementation Details and Metrics

XGBoost: 300 trees, depth 8, learning rate 0.1, subsampling 0.8, `multi:softprob`. LightGBM: up to 500 trees, 63 leaves, learning rate 0.05, early stopping at iteration 186. Random Forest (100 trees) and Logistic Regression (300 iterations) were trained on a stratified subsample of 500,007 resampled rows. The FT-style Transformer was trained for 10 epochs on CPU (about 1,085 s). SHAP used `TreeExplainer` on 2,000 random test flows; LIME used 2,000 perturbations per flow (1,500 in the v3 audit). The seed is 42 throughout. Because 82.8 % of test flows are benign, accuracy is uninformative; we report per-class precision/recall/F1 [25], macro-F1 (headline), macro one-vs-rest ROC-AUC, row-normalised confusion matrices and a benign-versus-attack view (false-alarm rate FP/(FP+TN), miss rate FN/(FN+TP)).

8. Results

8.1 Primary implementation

Detection. All tree models exceed 99.7 % accuracy but macro-F1 is only 0.78–0.83 (Table 5, Figures 3–4); Logistic Regression fails on Bot and all web attacks, confirming a non-linear problem. XGBoost's macro one-vs-rest ROC-AUC is 0.9996. In the binary view it detects 85,066 of 85,134 attack flows (miss rate 0.080 %) and flags 619 of 410,058 benign flows (false-alarm rate 0.151 %; precision 0.9928, F1 0.9960, PR-AUC 0.99986): detecting attacks is nearly solved, *naming* them is not. Nine of 15 classes reach F1 ≥ 0.95; macro-F1 is 0.892 over the 12 classes with at least 130 test flows and 0.978 over the ten non-web ones. The headline value is driven by (i) **web-attack confusion**: of 294 Brute-Force flows 112 are predicted as XSS and of 130 XSS flows 53 as Brute Force (flow statistics hold no HTTP payload); (ii) **false alarms**: of 619 misclassified benign flows 178 become PortScan, 162 Bot and 131 XSS, which explains the low precision of Bot (0.697) and XSS (0.238); (iii) **tiny classes**: Heartbleed (1 test flow), Infiltration (7) and SQL injection (4) give statistically meaningless metrics.

Table 5. Per-class test F1 (495,192 flows) for each model, and XGBoost under the per-class chronological split (XGB-chr).

Class	Support	XGB	LGBM	RF	LR	XGB-chr
BENIGN	410,058	0.9992	0.9990	0.9991	0.9898	0.9795
Bot	391	0.8043	0.8068	0.8235	0.0000	0.1323
DDoS	25,600	0.9997	0.9996	0.9996	0.9928	0.8346
DoS GoldenEye	2,055	0.9976	0.9956	0.9932	0.8145	0.9698
DoS Hulk	34,538	0.9984	0.9983	0.9970	0.9377	0.9683
DoS Slowhttptest	1,046	0.9914	0.9914	0.9890	0.6884	0.8792
DoS slowloris	1,075	0.9935	0.9944	0.9949	0.5634	0.8748
FTP-Patator	1,186	0.9996	1.0000	0.9996	0.9408	0.9889
Heartbleed	1	0.6667	0.6667	1.0000	0.5000	0.0000
Infiltration	7	0.7273	0.4211	0.4444	0.1270	0.7143
PortScan	18,164	0.9949	0.9949	0.9902	0.9781	0.9708
SSH-Patator	643	1.0000	1.0000	1.0000	0.8891	0.9851
Web – Brute Force	294	0.5914	0.5561	0.6716	0.0000	0.5359
Web – SQL Injection	4	0.4000	0.0000	0.0000	0.0000	0.7500
Web – XSS	130	0.3400	0.3074	0.4342	0.0000	0.2652
Macro-F1 (accuracy)		**0.8336 (0.9982)**	**0.7821 (0.9980)**	**0.8224 (0.9981)**	**0.5614 (0.9824)**	**0.7232 (0.9635)**

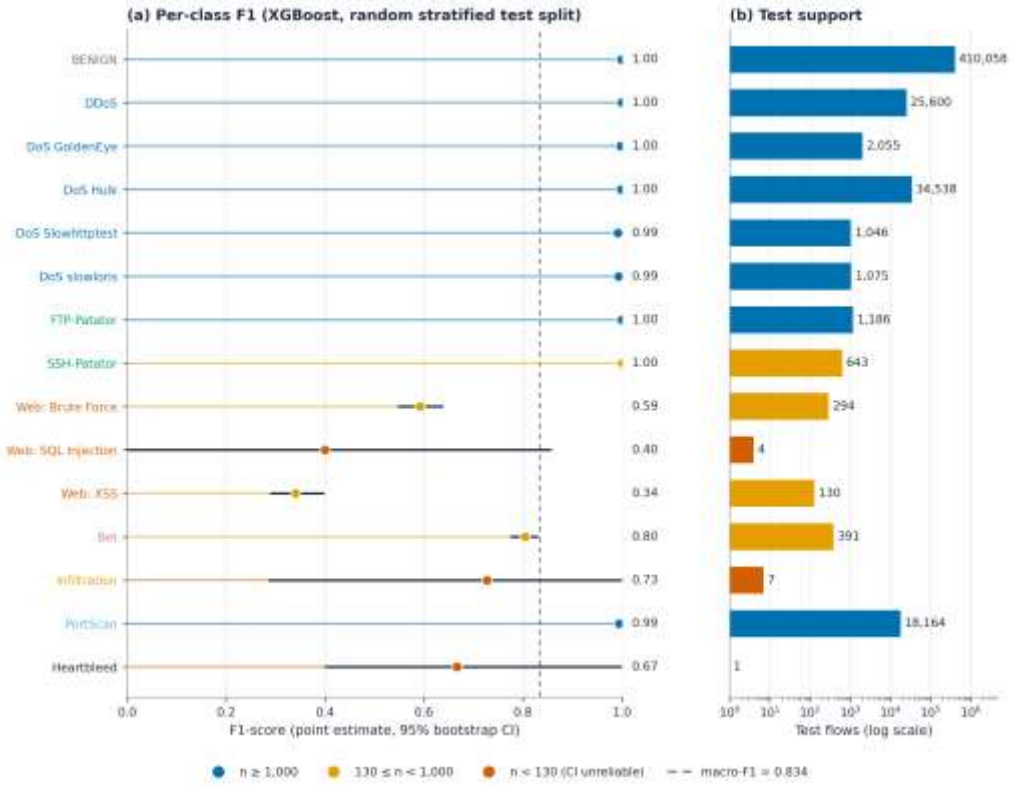


Figure 3. Per-class F1 of the compared models on the 495,192-flow test set; the web attacks and tiny classes are the weak spots.

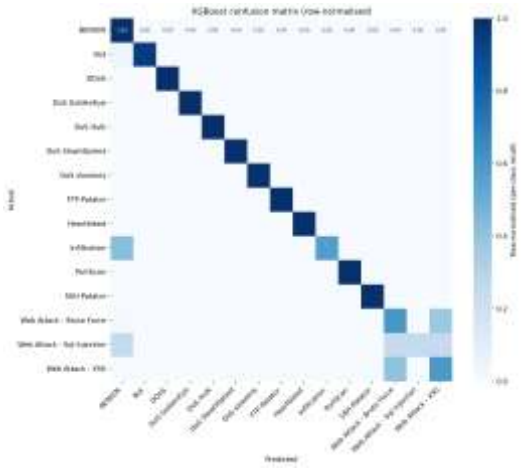


Figure 4. Row-normalised confusion matrix of XGBoost (rows = true class; each row sums to 1).

Over-fitting and time. On the natural training split XGBoost reaches accuracy 0.9984 and macro-F1 0.9238 against 0.9982 and 0.8336 on test (gap 0.090, from rare classes); stratified 5-fold macro-F1 is 0.870 (SD 0.017); the learning curve flattens above about 0.8 M flows (Figure 5a), so the bottleneck is the number of independent minority-class examples. Under the per-class chronological split macro-F1 falls to 0.723 (Figure 5b) and accuracy to 0.9635, 3.7 % of later benign flows are flagged and Bot precision collapses to 0.071, in line with the finding that random splits overstate performance [16, 56].

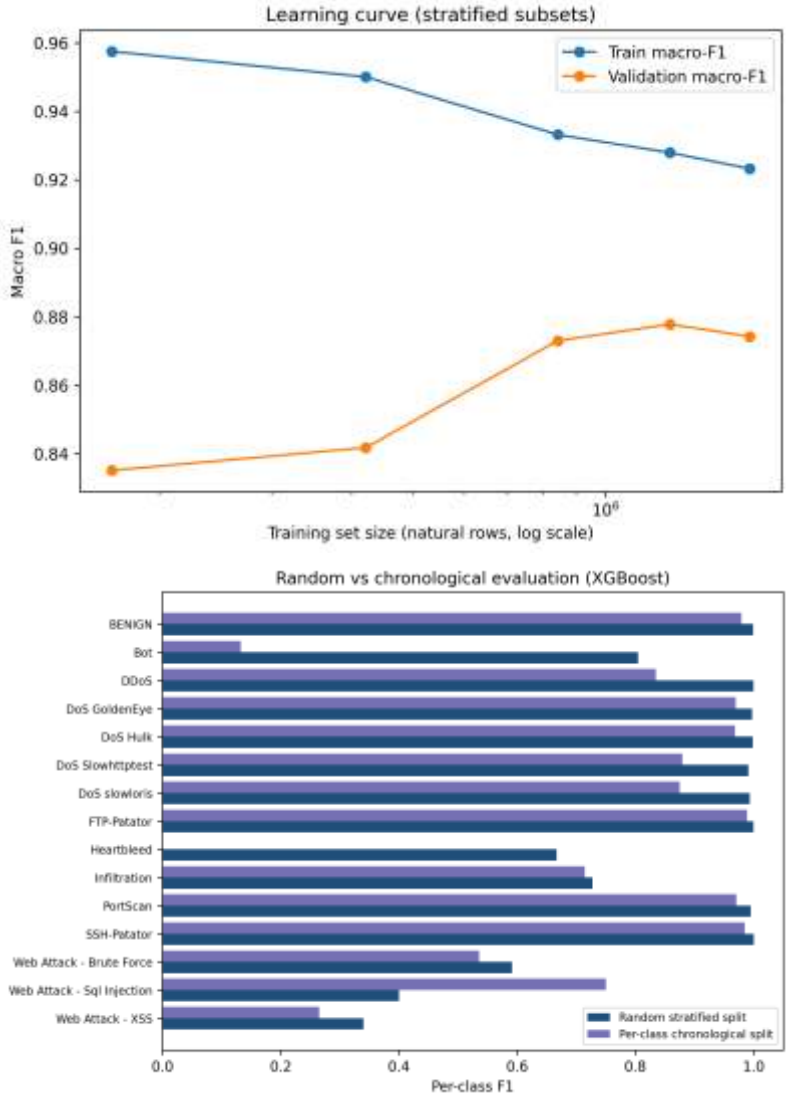


Figure 5. Over-fitting and time: (a) learning curve of XGBoost; (b) random versus per-class chronological split.

Port ablation and shortcut case. `Destination Port` is a legitimate behavioural feature but also a testbed artefact (FTP-Patator = port 21, SSH-Patator = port 22). Retraining XGBoost without it gives accuracy 0.9979 and macro-F1 0.8540 (final configuration: 0.8336), with Patator F1 still near 1.0; the model does not need the port because redundant timing and size features carry the same information, even though SHAP assigns it up to 14.7 log-odds: *attribution measures what the model uses, not what it needs.* The earlier configuration containing `Day_Index` reached 99.94 % accuracy and gave no warning, but SHAP ranked it first (mean attribution about 43 against 15 for the next feature) and produced the meaningless evidence "abnormally high day_index".

SHAP and LIME. Summed over classes, `Destination Port` has the largest SHAP attribution, followed by `Init_Win_bytes_forward`, `Init_Win_bytes_backward`, `Fwd IAT Std` and `Flow Duration`. Per class, Patator attacks are explained mostly by the port (mean |SHAP| 14.67 and 11.92), DoS Hulk by `Fwd IAT Std` and `Bwd Packet Length Std`, DDoS by `Fwd Packet Length Max`, PortScan by `Total Length of Fwd Packets` and web attacks by the backward window size. For the FTP-Patator alert (confidence 1.0000) the port contributes +14.74, followed by `Avg Fwd Segment Size` (+0.95) and `Flow Bytes/s` (+0.83); LIME gives ten tiny positive rules (≤ 0.0058) and does not report the port at all (Figure 6, which shows the SHAP and LIME evidence behind an analyst card), plausibly because saturation at probability ≈ 1 flattens the surrogate (not tested). Over 29 predicted-attack flows mean overlap@5 was 0.31, sign agreement 0.90 and top-1 agreement 13.8 %. The greedy counterfactual always succeeded (median 1 iteration, median L2 1.25) but often reached a benign probability of only 0.51–0.95. The generated seven-stage report for the FTP-Patator alert reads: *ATTACK DETECTED: FTP-Patator. Why? → Evidence: targets port 21 (FTP); abnormally low avg fwd segment size; abnormally low data transfer rate. Attack behaviour: repeated FTP login attempts consistent with credential guessing. Confidence: 100.00 %. Recommended response: lock/rate-limit the source IP; enforce account lockout; enable MFA on FTP.*

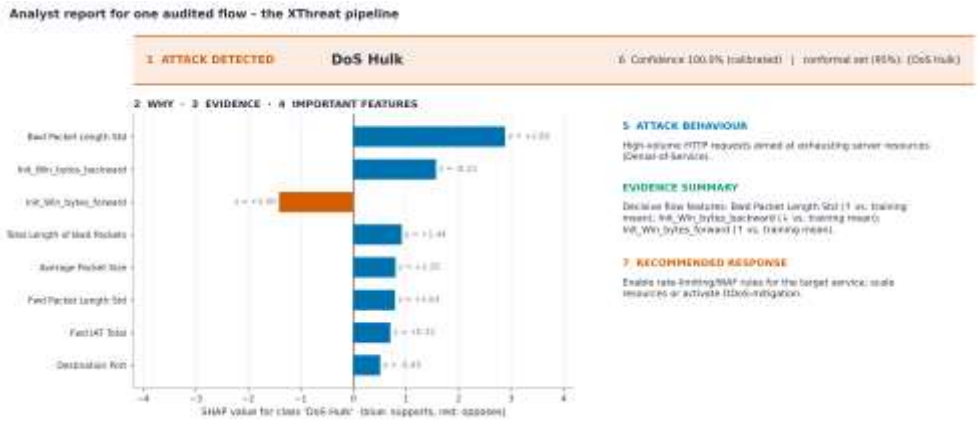


Figure 6. Seven-stage analyst report card generated for an FTP-Patator alert from SHAP, LIME and rule-based evidence.

8.2 Updated (v3) experiments

Paired bootstrap (21.1). Resampling the same test rows for two models gives a paired CI for the macro-F1 difference (Table 6, Figure 7). LightGBM (−0.047, CI −0.098 to +0.001) and Random Forest (−0.021, CI −0.076 to +0.030) are not significantly different from XGBoost, whereas Logistic Regression is clearly worse. McNemar's test flags small accuracy differences because of the large sample, so the macro-F1 interval is the more informative statistic.

Table 6. Paired bootstrap on the main split (1,000 replicates); FT-Transformer from cell 21.8.

Model	Macro-F1	95 % CI	MCC	ΔF1 vs. XGB (paired CI)	McNemar p
XGBoost	0.8336	[0.785, 0.887]	0.9942	–	–
LightGBM	0.7821	[0.759, 0.818]	0.9933	−0.047 [−0.098, +0.001]	9.6e-09
Random Forest	0.8224	[0.777, 0.848]	0.9936	−0.021 [−0.076, +0.030]	8.9e-03

Logistic Regression	0.5614	[0.548, 0.600]	0.9420	−0.267 [−0.319, −0.217]	< 1e-300
FT-Transformer	0.6697	–	0.9750	−0.153 [−0.215, −0.099]	–

Multi-seed (21.2). Over seeds 11, 22 and 33 macro-F1 is 0.845 ± 0.018 for XGBoost (95 % t-interval half-width 0.045), 0.824 ± 0.012 for Random Forest and 0.810 ± 0.016 for LightGBM; the ranking changes with the seed (in seed 22 LightGBM leads), consistent with the paired bootstrap. Per-class SDs are near zero for large classes and large for Infiltration (0.56 ± 0.28). Mutual-information selection was perfectly stable (mean pairwise Jaccard 1.000, the same 30 features in every seed).

Near-duplicate leakage (21.3). In the random split 51.4 % of test flows have a near-identical training flow at one decimal of rounding and 73.9 % at zero decimals; for DoS and web-attack classes the share is 95–99 % (DoS Hulk 0.809/0.993, slowloris 0.954/0.988, Brute Force 0.969/0.983). The flows form 730,938 near-duplicate groups. Under the group-aware split XGBoost reaches accuracy 0.9985 and macro-F1 0.864 over the classes present in the test set (Heartbleed is absent) against 0.846 for the random split over the same classes, so the high near-duplicate rate did *not* lower macro-F1 under grouping; per-class values are not directly comparable because supports change. The diagnostic still shows that a random split lets most flows of several classes be classified by lookup of a sibling.

Imbalance ablation (21.4). Natural unweighted training gives the highest macro-F1 (0.892), above capped SMOTE with tiny-class weights (0.834, the configuration used in the main results) and sqrt-inverse weights (0.828). The 0.058 difference is explained almost entirely by three classes with 1, 4 and 7 test flows (Heartbleed 1.000 vs. 0.667, SQL injection 0.857 vs. 0.400, Infiltration 0.833 vs. 0.727); Bot, Brute Force and XSS change little (0.828 vs. 0.804, 0.662 vs. 0.591, 0.216 vs. 0.340). This single-run evidence does not prove SMOTE harmful, but the data provide *no evidence that SMOTE helps*; the capped SMOTE policy should be presented as a design decision, not a demonstrated improvement.

Calibration and conformal sets (21.5). Raw XGBoost probabilities are already well calibrated (ECE 0.00049, Brier 0.00256, NLL 0.00508; after temperature scaling with $T = 1.07$: 0.00046, 0.00255, 0.00502), although this is dominated by the benign class. Conformal sets at $\alpha = 0.05$ reach marginal coverage 0.998 with average set size 1.000: 99.88 % of flows receive a single label (accuracy 0.999) and 0.12 % are ambiguous (accuracy only 0.364) and should go to an analyst. Marginal coverage hides class-level failures (Brute Force 0.592, XSS 0.615), which the class-conditional variant repairs (0.990, 0.989); SQL injection (0.000) and Infiltration (0.400) stay uncovered because they have fewer than 20 calibration flows. On the chronological split the same thresholds give coverage 0.9635 (still above the nominal 0.95 but down from 0.998; minimum class coverage remains 0), so exchangeability should not be assumed under temporal shift (Figure 8).

Explanation audit (21.6). On 55 predicted-attack flows from 11 classes, SHAP–LIME overlap@5 is 0.273 (95 % CI 0.218–0.324) against chance 0.167, overlap@3 0.242 (chance 0.100), Spearman correlation of |attributions| only 0.141 and sign agreement among shared top-5 features 0.880 (chance 0.5). Merging the 30 features into 13 correlated clusters raises overlap@3 to 0.509 (chance 0.231): the methods often name different but correlated features (credit splitting). LIME is only moderately self-consistent (overlap@5 0.411 across seeds). Deletion fidelity favours SHAP: AOPC 0.806 for SHAP, 0.589 for LIME, 0.608 for the global SHAP order and 0.254 for random; sufficiency of the top-5 features is 0.145 (SHAP), 0.046 (LIME), 0 (random). Under input noise the SHAP top-5 Jaccard is 0.602 and Spearman 0.855. Replacing features by the benign mean may create off-manifold inputs, so fidelity values are relative comparisons.

Constrained counterfactuals (21.7). The unconstrained greedy search is always valid but changes 10 features on average, changes the port in 18.2 % of flows and yields no plausible counterfactual (L1 5.62, L2 1.82, benign probability 0.985). The constrained, pruned search is also 100 % valid, changes 2

features, never the port, and has L1 1.09 and L2 0.91; 27.3 % of its counterfactuals are within the benign-neighbourhood threshold (kNN distance 0.660), at the price of a lower benign probability after the flip (0.699): they sit close to the decision boundary and are probes of the model, not advice an attacker could follow.

Transformer and cost (21.8). The FT-style Transformer reaches accuracy 0.9922 and macro-F1 0.670 against 0.9982 and 0.834 for XGBoost (paired difference −0.153, 95 % CI −0.215 to −0.099) and has a worse ECE (0.0024 vs. 0.0005). Integrated Gradients satisfy completeness approximately (median error 0.031, maximum 0.545) and beat random deletion (AOPC 0.844 vs. 0.572), but agree with XGBoost's SHAP only at chance level (overlap@5 0.185 vs. 0.167 on 53 flows both models label identically): the two models rely on different features, in line with [4, 26]. The Transformer was trained for 10 epochs without tuning, so it is a baseline. Detection costs 0.0099 ms per flow (about 101,000 flows/s), TreeSHAP 3.7 ms and LIME 61.5 ms, so TreeSHAP can explain every alert in real time while LIME suits on-demand use. **Cross-dataset (21.9):** not run; generalisation beyond CIC-IDS2017 is untested.

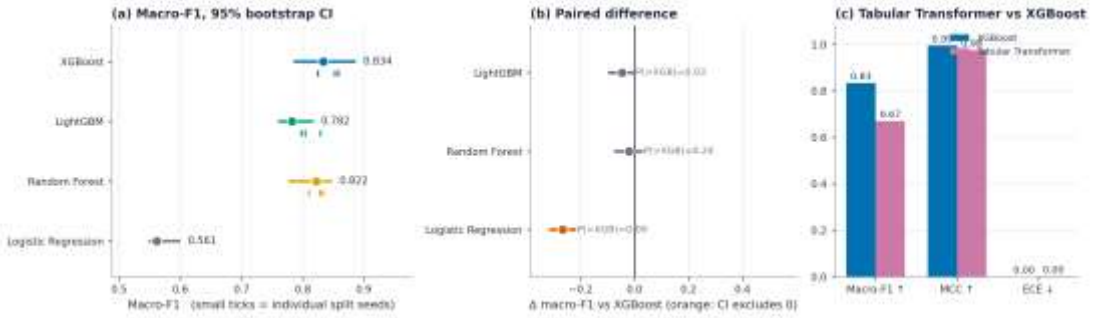


Figure 7. Model comparison with bootstrap CIs, paired differences and the Transformer.

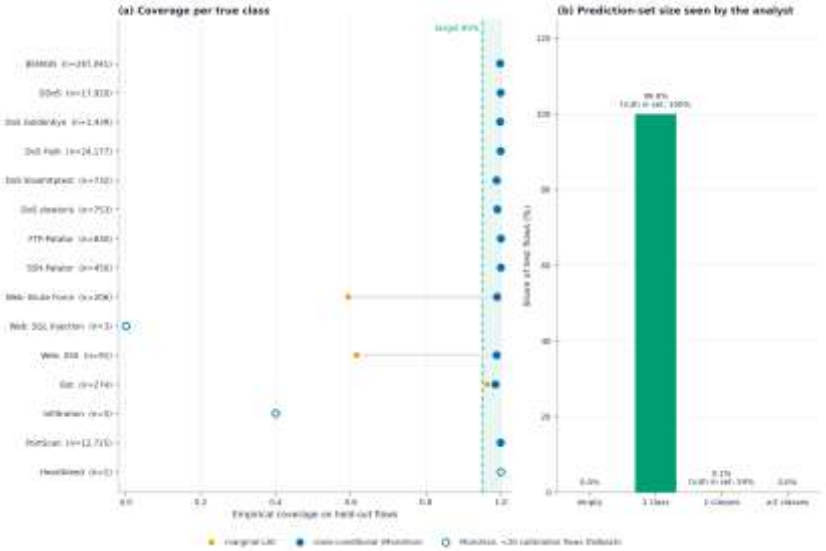


Figure 8. Conformal coverage and prediction-set sizes: marginal versus class-conditional, and under the chronological split.

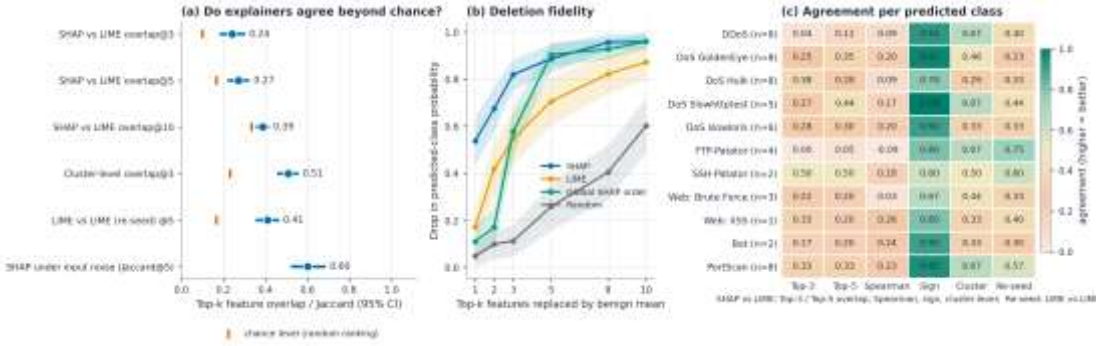


Figure 9. Explanation audit: SHAP–LIME agreement against chance, deletion fidelity and stability.

9. Discussion

What the update changes. Because Sections 1–20 are unchanged, the primary findings stand: the detector separates benign from attack almost perfectly, struggles to name web attacks, is sensitive to the split and its explanations disagree across methods. v3 turns each observation into a tested claim and refines it: agreement is above chance but low, the disagreement is partly credit splitting between correlated features, and SHAP is more faithful than LIME under deletion. Some earlier design choices are *not* supported by the new evidence: SMOTE showed no benefit, the group-aware split did not lower macro-F1, and "XGBoost is best" holds only against Logistic Regression and the Transformer. Raw probabilities were already calibrated, so the value of the conformal stage lies in flagging the 0.12 % ambiguous flows and in exposing that marginal coverage hides failures on web attacks.

Relation to the literature. Srisumrith and Sodsee [57] report macro-F1 0.997 on a 20 % sample with random splits against our 0.834; the difference cannot be attributed to one cause, but the score depends strongly on the class set (0.892 over 12 classes, 0.978 without web attacks), on de-duplication and on rare-class support, and 51–74 % of random-split test flows have a sibling in training. The drop to 0.723 under a chronological split points the same way as Moczkodan and Ragab [56]. Low explainer agreement is consistent with [10–12]; our audit adds that SHAP and LIME agree on the sign (0.88) and agree much better once correlated features are merged. Hermosilla et al. [53] find SHAP and LIME complementary; with SHAP clearly more faithful and LIME only moderately self-consistent, our evidence favours SHAP as primary explainer and LIME as a cross-check, and agreement is not faithfulness [54]. Constraints matter for counterfactuals [37, 55], yet only 27 % of ours were plausible. Finally, the shortcut case confirms the warning on spurious correlations [16]: a provenance feature was invisible in accuracy but dominated the attributions.

10. Limitations

- **No cross-dataset test:** cell 21.9 was skipped; generalisation beyond CIC-IDS2017 is untested.
- **Deep models:** the CSV release has no IP addresses or timestamps, so sequence models and graphs cannot be built; the Transformer is a feature-token model trained for 10 epochs without tuning and the GNN remains design-only.
- **Tiny classes and few seeds:** Heartbleed (1 test flow), SQL injection (4) and Infiltration (7) make several results fragile; multi-seed results use three seeds; the chronological split is a proxy because row order is not a timestamp.
- **Internal and construct validity:** median imputation and the Isolation-Forest filter were applied before the split (about 0.1 % of rows and 1 % of benign rows); mutual-information selection ignores redundancy; the fidelity test may create off-manifold inputs; the evidence layer is template-based and no user study measured analyst trust.
- **Update-plan sources:** the five AI review reports were not available when this document was written; the plan is reconstructed from the concern table in the v3 notebook.

11. Conclusion and Future Work

The project implements the instructor's seven-stage XThreat pipeline on CIC-IDS2017 and revises it critically. XGBoost reaches 99.82 % accuracy and macro-F1 0.834 (benign-versus-attack F1 0.996, false-alarm rate 0.15 %), falling to 0.723 under a chronological split. The v3 update shows that the tree ensembles are statistically inseparable, that random-split test sets are full of near-duplicates, that SMOTE brought no measurable benefit, that probabilities are well calibrated and conformal sets can route 0.12 % of flows to analysts, that SHAP and LIME agree above chance but weakly and SHAP is more faithful, and that constrained counterfactuals are sparser and respect the immutable port but are only 27 %

plausible. The earlier lessons remain: attribution to the destination port does not mean the model needs it, and explanations can expose shortcut features that accuracy conceals. Explainability should be used to validate IDS models, not only to communicate their outputs.

Future work: (1) run the cross-dataset test on CSE-CIC-IDS2018 and on corrected CIC-IDS2017 releases with timestamps; (2) build flow sequences and host graphs for sequence Transformers and GNNs with matching explainers; (3) improve counterfactual plausibility with diversity or generative constraints [37]; (4) validate evidence and response templates with SOC analysts; (5) report rare classes with intervals and test further minority-class strategies.

Use of AI tools. AI-assisted tools were used for code debugging, LaTeX/document formatting, plotting scripts, the five AI review reports required by the instructor, and for drafting and structuring this report from the author's notebooks and paper. The author is responsible for checking the content, figures and conclusions. *(Edit to match the course's AI-use policy.)*

References

[1] I. Sharafaldin, A. H. Lashkari and A. A. Ghorbani, "Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization," *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*, pp. 108–116, 2018. https://doi.org/10.5220/0006639801080116

[2] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785–794, 2016. https://doi.org/10.1145/2939672.2939785

[3] G. Ke et al., "LightGBM: A Highly Efficient Gradient Boosting Decision Tree," *Advances in Neural Information Processing Systems 30 (NeurIPS)*, pp. 3146–3154, 2017. https://proceedings.neurips.cc/paper_files/paper/2017/hash/6449f44a102fde848669bdd9eb6b76fa-Abstract.html

[4] L. Grinsztajn, E. Oyallon and G. Varoquaux, "Why Do Tree-Based Models Still Outperform Deep Learning on Typical Tabular Data?," *Advances in Neural Information Processing Systems 35, Datasets and Benchmarks Track*, 2022. https://datasets-benchmarks-proceedings.neurips.cc/paper_files/paper/2022/hash/0349c13225a829a256f1b0e2bc77478e-Abstract-Datasets_and_Benchmarks.html

[5] F. Doshi-Velez and B. Kim, "Towards A Rigorous Science of Interpretable Machine Learning," *arXiv preprint arXiv:1702.08608*, 2017. https://doi.org/10.48550/arXiv.1702.08608

[6] Z. C. Lipton, "The Mythos of Model Interpretability," *Communications of the ACM*, vol. 61, no. 10, pp. 36–43, 2018. https://doi.org/10.1145/3233231

[7] T. Miller, "Explanation in Artificial Intelligence: Insights from the Social Sciences," *Artificial Intelligence*, vol. 267, pp. 1–38, 2019. https://doi.org/10.1016/j.artint.2018.07.007

[8] H. S. Eriksson and G. Grov, "Towards XAI in the SOC – a user centric study of explainable alerts with SHAP and LIME," 2022 IEEE International Conference on Big Data (Big Data), Osaka, Japan, 2022, pp. 2595–2600, https://doi.org/10.1109/BigData55660.2022.10020248.

[9] N. Rastogi et al., "Too Much to Trust? Measuring the Security and Cognitive Impacts of Explainability in AI-Driven SOCs," *arXiv preprint arXiv:2503.02065*, 2025. https://doi.org/10.48550/arXiv.2503.02065

[10] S. Krishna et al., "The Disagreement Problem in Explainable Machine Learning: A Practitioner's Perspective," *Transactions on Machine Learning Research*, 2024. https://doi.org/10.48550/arXiv.2202.01602

[11] K. Dietz et al., "Agree to Disagree: Exploring Consensus of XAI Methods for ML-based NIDS," *2024 20th International Conference on Network and Service Management (CNSM)*, pp. 1–7, 2024. https://doi.org/10.23919/CNSM62983.2024.10814448

[12] A. Kumar and V. L. L. Thing, "Evaluating the Explainability of State-of-the-Art Deep Learning-based Network Intrusion Detection Systems," *arXiv preprint arXiv:2408.14040*, 2025. https://doi.org/10.48550/arXiv.2408.14040

[13] A. L. Buczak and E. Guven, "A Survey of Data Mining and Machine Learning Methods for Cyber Security Intrusion Detection," *IEEE Communications Surveys & Tutorials*, vol. 18, no. 2, pp. 1153–1176, 2016. https://doi.org/10.1109/COMST.2015.2499621

[14] R. Vinayakumar et al., "Deep Learning Approach for Intelligent Intrusion Detection System," *IEEE Access*, vol. 7, pp. 41525–41550, 2019. https://doi.org/10.1109/ACCESS.2019.2929570

[15] R. Sommer and V. Paxson, "Outside the Closed World: On Using Machine Learning for Network Intrusion Detection," *2010 IEEE Symposium on Security and Privacy*, pp. 305–316, 2010. https://doi.org/10.1109/SP.2010.25

[16] D. Arp et al., “Dos and Don'ts of Machine Learning in Computer Security,” *31st USENIX Security Symposium*, pp. 3971–3988, 2022. https://www.usenix.org/conference/usenixsecurity22/presentation/arp

[17] R. Panigrahi and S. Borah, “A Detailed Analysis of CICIDS2017 Dataset for Designing Intrusion Detection Systems,” *International Journal of Engineering & Technology*, vol. 7, no. 3.24, pp. 479–482, 2018.

[18] Z. K. Maseer et al., “Benchmarking of Machine Learning for Anomaly Based Intrusion Detection Systems in the CICIDS2017 Dataset,” *IEEE Access*, vol. 9, pp. 22351–22370, 2021. https://doi.org/10.1109/ACCESS.2021.3054953

[19] M. Ring et al., “A Survey of Network-based Intrusion Detection Data Sets,” *Computers & Security*, vol. 86, pp. 147–167, 2019. https://doi.org/10.1016/j.cose.2019.01.009

[20] G. Engelen, V. Rimmer and W. Joosen, “Troubleshooting an Intrusion Detection Dataset: the CICIDS2017 Case Study,” *2021 IEEE Security and Privacy Workshops (SPW)*, pp. 7–12, 2021. https://doi.org/10.1109/SPW53761.2021.00009

[21] M. Lanvin et al., “Errors in the CICIDS2017 Dataset and the Significant Differences in Detection Performances It Makes,” *Risks and Security of Internet and Systems*, vol. 13857, 2023. https://doi.org/10.1007/978-3-031-31108-6_2

[22] H. He and E. A. Garcia, “Learning from Imbalanced Data,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 21, no. 9, pp. 1263–1284, 2009. https://doi.org/10.1109/TKDE.2008.239

[23] N. V. Chawla et al., “SMOTE: Synthetic Minority Over-sampling Technique,” *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002. https://doi.org/10.1613/jair.953

[24] G. Lemaître, F. Nogueira and C. K. Aridas, “Imbalanced-learn: A Python Toolbox to Tackle the Curse of Imbalanced Datasets in Machine Learning,” *Journal of Machine Learning Research*, vol. 18, no. 17, pp. 1–5, 2017. https://jmlr.org/papers/v18/17-365.html

[25] M. Sokolova and G. Lapalme, “A Systematic Analysis of Performance Measures for Classification Tasks,” *Information Processing & Management*, vol. 45, no. 4, pp. 427–437, 2009. https://doi.org/10.1016/j.ipm.2009.03.002

[26] R. Shwartz-Ziv and A. Armon, “Tabular Data: Deep Learning is Not All You Need,” *Information Fusion*, vol. 81, pp. 84–90, 2022. https://doi.org/10.1016/j.inffus.2021.11.026

[27] A. Vaswani et al., “Attention Is All You Need,” *Advances in Neural Information Processing Systems 30 (NeurIPS)*, pp. 5998–6008, 2017. https://proceedings.neurips.cc/paper_files/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html

[28] Y. Gorishniy et al., “Revisiting Deep Learning Models for Tabular Data,” *Advances in Neural Information Processing Systems 34 (NeurIPS)*, pp. 18932–18943, 2021. https://proceedings.neurips.cc/paper_files/paper/2021/hash/9d86d83f825a20c1038d7b30d6e7bf78-Abstract.html

[29] T. N. Kipf and M. Welling, “Semi-Supervised Classification with Graph Convolutional Networks,” *International Conference on Learning Representations (ICLR)*, 2017. https://doi.org/10.48550/arXiv.1609.02907

[30] W. L. Hamilton, R. Ying and J. Leskovec, “Inductive Representation Learning on Large Graphs,” *Advances in Neural Information Processing Systems 30 (NeurIPS)*, pp. 1024–1034, 2017. https://proceedings.neurips.cc/paper_files/paper/2017/hash/5dd9db5e033da9c6fb5ba83c7a7ebea9-Abstract.html

[31] W. W. Lo et al., “E-GraphSAGE: A Graph Neural Network based Intrusion Detection System for IoT,” *NOMS 2022 – IEEE/IFIP Network Operations and Management Symposium*, pp. 1–9, 2022. https://doi.org/10.1109/NOMS54207.2022.9789878

[32] A. Barredo Arrieta et al., “Explainable Artificial Intelligence (XAI): Concepts, Taxonomies, Opportunities and Challenges toward Responsible AI,” *Information Fusion*, vol. 58, pp. 82–115, 2020. https://doi.org/10.1016/j.inffus.2019.12.012

[33] S. M. Lundberg and S. I. Lee, “A Unified Approach to Interpreting Model Predictions,” *Advances in Neural Information Processing Systems 30 (NeurIPS)*, pp. 4765–4774, 2017. https://proceedings.neurips.cc/paper_files/paper/2017/hash/8a20a8621978632d76c43dfd28b67767-Abstract.html

[34] S. M. Lundberg et al., “From Local Explanations to Global Understanding with Explainable AI for Trees,” *Nature Machine Intelligence*, vol. 2, no. 1, pp. 56–67, 2020. https://doi.org/10.1038/s42256-019-0138-9

[35] M. T. Ribeiro, S. Singh and C. Guestrin, “``Why Should I Trust You?": Explaining the Predictions of Any Classifier,” *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 1135–1144, 2016. https://doi.org/10.1145/2939672.2939778

[36] S. Wachter, B. Mittelstadt and C. Russell, “Counterfactual Explanations without Opening the Black Box: Automated Decisions and the GDPR,” *Harvard Journal of Law & Technology*, vol. 31, no. 2, pp. 841–887, 2018. https://jolt.law.harvard.edu/assets/articlePDFs/v31/WachterMittelstadtRussell-CounterfactualExplanations.pdf

[37] R. K. Mothilal, A. Sharma and C. Tan, “Explaining Machine Learning Classifiers through Diverse Counterfactual Explanations,” *Proceedings of the 2020 Conference on Fairness, Accountability, and Transparency (FAT*)*, pp. 607–617, 2020. https://doi.org/10.1145/3351095.3372836

[38] M. Sundararajan, A. Taly and Q. Yan, "Axiomatic Attribution for Deep Networks," *Proceedings of the 34th International Conference on Machine Learning (ICML)*, vol. 70, pp. 3319–3328, 2017. https://proceedings.mlr.press/v70/sundararajan17a.html

[39] D. Slack et al., "Fooling LIME and SHAP: Adversarial Attacks on Post hoc Explanation Methods," *Proceedings of the AAAI/ACM Conference on AI, Ethics, and Society (AIES)*, pp. 180–186, 2020. https://doi.org/10.1145/3375627.3375818

[40] D. Alvarez-Melis and T. S. Jaakkola, "On the Robustness of Interpretability Methods," *arXiv preprint arXiv:1806.08049*, 2018. https://doi.org/10.48550/arXiv.1806.08049

[41] W. Samek et al., "Evaluating the Visualization of What a Deep Neural Network Has Learned," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 28, no. 11, pp. 2660–2673, 2017. https://doi.org/10.1109/TNNLS.2016.2599820

[42] C. Guo et al., "On Calibration of Modern Neural Networks," *Proceedings of the 34th International Conference on Machine Learning (ICML)*, vol. 70, pp. 1321–1330, 2017. https://proceedings.mlr.press/v70/guo17a.html

[43] V. Vovk, A. Gammerman and G. Shafer, "Algorithmic Learning in a Random World," *Springer*, 2005. https://doi.org/10.1007/b106715

[44] A. N. Angelopoulos and S. Bates, "Conformal Prediction: A Gentle Introduction," *Foundations and Trends in Machine Learning*, vol. 16, no. 4, pp. 494–591, 2023. https://doi.org/10.1561/2200000101

[45] B. Efron and R. J. Tibshirani, "An Introduction to the Bootstrap," *Chapman & Hall*, 1993. https://doi.org/10.1007/978-1-4899-4541-9

[46] Q. McNemar, "Note on the Sampling Error of the Difference Between Correlated Proportions or Percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947. https://doi.org/10.1007/BF02295996

[47] B. W. Matthews, "Comparison of the Predicted and Observed Secondary Structure of T4 Phage Lysozyme," *Biochimica et Biophysica Acta (BBA) – Protein Structure*, vol. 405, no. 2, pp. 442–451, 1975. https://doi.org/10.1016/0005-2795(75)90109-9

[48] D. L. Marino, C. S. Wickramasinghe and M. Manic, "An Adversarial Approach for Explainable AI in Intrusion Detection Systems," *IECON 2018 – 44th Annual Conference of the IEEE Industrial Electronics Society*, pp. 3237–3243, 2018. https://doi.org/10.1109/IECON.2018.8591457

[49] M. Wang et al., "An Explainable Machine Learning Framework for Intrusion Detection Systems," *IEEE Access*, vol. 8, pp. 73127–73141, 2020. https://doi.org/10.1109/ACCESS.2020.2988359

[50] S. Mane and D. Rao, "Explaining Network Intrusion Detection System Using Explainable AI Framework," *arXiv preprint arXiv:2103.07110*, 2021. https://doi.org/10.48550/arXiv.2103.07110

[51] S. Neupane et al., "Explainable Intrusion Detection Systems (X-IDS): A Survey of Current Methods, Challenges, and Opportunities," *IEEE Access*, vol. 10, pp. 112392–112415, 2022. https://doi.org/10.48550/arXiv.2207.06236

[52] A. Warnecke et al., "Evaluating Explanation Methods for Deep Learning in Security," *2020 IEEE European Symposium on Security and Privacy (EuroS&P)*, pp. 158–174, 2020. https://doi.org/10.1109/EuroSP48549.2020.00018

[53] P. Hermosilla, S. Berríos and H. Allende-Cid, "Explainable AI for Forensic Analysis: A Comparative Study of SHAP and LIME in Intrusion Detection Models," *Applied Sciences*, vol. 15, no. 13, art. no. 7329, 2025. https://doi.org/10.3390/app15137329

[54] R. Kalakoti et al., "Evaluating Explainable AI for Deep Learning-Based Network Intrusion Detection System Alert Classification," *arXiv preprint arXiv:2506.07882*, 2025. https://doi.org/10.48550/arXiv.2506.07882

[55] V. Galwaduge and J. Samarabandu, "Novel Actionable Counterfactual Explanations for Intrusion Detection Using Diffusion Models," *Journal of Cybersecurity and Privacy*, vol. 5, no. 3, art. no. 68, 2025. https://doi.org/10.3390/jcp5030068

[56] Z. Moczkodan and H. Ragab, "Do Transformers Actually Help Intrusion Detection? A Temporal Sequence Evaluation on CIC-IDS2017," *arXiv preprint arXiv:2606.11098*, 2026. https://doi.org/10.48550/arXiv.2606.11098

[57] N. Srisumrith and S. Sodsee, "Detecting Cybersecurity Threats by Integrating Explainable AI with SHAP Interpretability and Strategic Data Sampling," *arXiv preprint arXiv:2602.19087*, 2026. https://doi.org/10.48550/arXiv.2602.19087

[58] F. T. Liu, K. M. Ting and Z. H. Zhou, "Isolation Forest," *2008 Eighth IEEE International Conference on Data Mining*, pp. 413–422, 2008. https://doi.org/10.1109/ICDM.2008.17

[59] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. https://jmlr.org/papers/v12/pedregosa11a.html